



UNIVERSITÀ DEGLI STUDI DI FIRENZE
SCUOLA DI INGEGNERIA - DIPARTIMENTO DI INGEGNERIA
DELL'INFORMAZIONE

Tesi di Laurea Triennale in Ingegneria Informatica

**SIMULAZIONE DI ROBOT MOBILE E
LOCALIZZAZIONE TRAMITE ALGORITMO ICP SU
SCANSIONI LIDAR**

Candidato
Pinzani Eros

Relatore
Prof. Battistelli Giorgio

N. Matricola
7030989

Anno Accademico 2024/2025

Indice

Elenco delle figure	iii
Elenco delle tabelle	iii
Abstract	i
Introduzione	ii
1 Fondamenti teorici e modello del sistema	1
1.1 Modello cinematico del robot mobile	1
1.2 Sensori LiDAR per la robotica mobile	3
1.3 Localizzazione robotica	4
1.4 Algoritmo ICP (Iterative Closest Point)	5
2 Implementazione del sistema di simulazione	7
2.1 Architettura del simulatore	8
2.2 Modellazione del robot	9
2.3 Ambiente e gestione degli ostacoli	9
2.4 Generazione delle traiettorie	11
2.5 Simulazione del sensore LiDAR	12
3 Algoritmo ICP e ricostruzione della posa	16

3.1	Implementazione dell'algoritmo ICP	16
3.2	ICP con e senza inizializzazione odometrica	19
3.3	Ricostruzione della traiettoria	21
3.4	Metriche di valutazione	22
4	Validazione sperimentale	24
5	Conclusioni e sviluppi futuri	25
	Bibliografia	26

Elenco delle figure

Elenco delle tabelle

Abstract

Il crescente impiego di robot mobili autonomi in ambienti dinamici e non strutturati richiede capacità avanzate di localizzazione e mappatura in tempo reale. L'algoritmo *Iterative Closest Point (ICP)* rappresenta una soluzione consolidata per l'allineamento di nuvole di punti provenienti da scansioni in istanti di tempo successivi, ricavate da un sensore *LiDAR*, permettendo una stima accurata della posa del robot.

In questo progetto si implementa e valuta un sistema di localizzazione 2D basato su ICP point-to-point, analizzando l'impatto di diverse strategie di filtraggio dei dati e tecniche di ottimizzazione. Sono state sviluppate e confrontate molteplici varianti algoritmiche integrate con filtri per la rimozione del rumore e l'eliminazione degli outlier. Le prestazioni del sistema sono state valutate attraverso metriche quantitative di errore di posizione e orientamento, nonché mediante un'analisi qualitativa della convergenza dell'algoritmo in scenari simulati con ostacoli geometrici di diversa complessità.

Introduzione

La robotica mobile ha un utilizzo molto diffuso in ambiti che spaziano dalla logistica industriale alla sorveglianza, dalla ricerca e soccorso all'esplorazione di ambienti ostili. Ma non solo, è ormai comune trovare piccoli robot mobili nella quotidianità; ad esempio, basti pensare ai robot aspirapolvere per la pulizia della casa.

Un elemento essenziale per l'autonomia operativa del robot mobile è la *localizzazione*, ovvero la capacità di stimare con precisione la propria posizione e orientamento nello spazio. In assenza di sistemi di posizionamento globale affidabili, come nei contesti di ambienti al chiuso o in cui la copertura GPS è limitata, la localizzazione deve necessariamente basarsi su sensori embedded e algoritmi di elaborazione dati in tempo reale.

Tra tali sensori, uno dei più diffusi per la percezione ambientale nei robot mobili è il **LiDAR** (*Light Detection and Ranging*). La sua capacità principale è quella di acquisire informazioni geometriche dettagliate dell'ambiente circostante sotto forma di nuvole di punti. Questi dati sono misurazioni di distanza lungo direzioni angolari diverse, fornendo una rappresentazione discreta accurata della struttura spaziale dell'ambiente. Tramite poi l'elaborazione di scansioni LiDAR successive si è in grado di stimare il movimento del robot tramite tecniche di registrazione geometrica, ovvero algoritmi che

determinano la trasformazione rigida (traslazione e rotazione) che allinea al meglio due nuvole di punti acquisite in istanti temporali consecutivi.

L'algoritmo **ICP** (*Iterative Closest Point*) costituisce uno degli approcci più diffusi per la registrazione di nuvole di punti. I passi compiuti da ICP si basano su un processo iterativo che alterna la ricerca delle corrispondenze punto a punto tra due scansioni adiacentie l'ottimizzazione della trasformazione che minimizza l'errore quadratico medio tra i punti corrispondenti. Nonostante l'efficacia dell'algoritmo in numerose applicazioni, l'implementazione pratica di ICP presenta diversi ostacoli: sensibilità alla presenza di outlier, dipendenza dalla stima iniziale della trasformazione, necessità di bilanciamento tra accuratezza ed efficienza computazionale.

Questo progetto vuole implementare e valutare un sistema di localizzazione 2D per robot mobili operanti in ambienti strutturati, utilizzando dati provenienti da un sensore LiDAR. L'obiettivo principale è integrare l'algoritmo ICP point to point con tecniche di pre elaborazione dei dati provenienti dal sensore, al fine di migliorare robustezza e accuratezza della stima di posa in presenza di rumore di misura e dinamiche ambientali complesse.

In particolare, il progetto si concentra sui seguenti aspetti:

- **Implementazione modulare dell'algoritmo ICP:** sviluppo di una versione point to point dell'algoritmo con attenzione all'efficienza computazionale e alla flessibilità nella configurazione dei parametri operativi.
- **Filtraggio dei dati:** analisi e implementazione di tecniche di pre elaborazione per la rimozione del rumore e l'identificazione di outlier.
- **Gestione di ambienti complessi:** creazione di un ambiente 2D con

la presenza di ostacoli geometrici di diversa tipologia (rettangoli, cerchi, poligoni, muri) che permetta di generare scansioni LiDAR utilizzando tecniche di ray casting per emulare il comportamento fisico del sensore.

- **Valutazione quantitativa e qualitativa:** definizione e calcolo di metriche di prestazione per la valutazione dell'errore di localizzazione in termini di posizione e orientamento, analisi visiva della convergenza dell'algoritmo e della qualità dell'allineamento ottenuto.

Il progetto è stato scritto con il linguaggio **Python**, utilizzando librerie per la computazione scientifica (**numpy**), la gestione di geometrie (**shapely**) e la visualizzazione dei risultati (**matplotlib**).

L'architettura adotta un approccio modulare che separa le responsabilità: la classe **Environment** gestisce la rappresentazione geometrica dell'ambiente, le classi **ICP** e **Lidar** si occupano della gestione dei compiti dell'algoritmo e del sensore; ci sono poi la classe **Robot** che rappresenta il robot con modello *uniciclo*, la classe **TrajectoryGenerator** che gestisce i comandi del robot in modo tale che segua traiettorie specifiche da utilizzare come *Ground Truth* e infine **Visualizer** che si occupa dell'interfaccia grafica. Inoltre **main** orchestra e unisce i vari metodi in modo tale da seguire una specifica pipeline che permette di ottenere i corretti risultati.

L'approccio utilizzato prevede una fase iniziale in cui vengono generati dati sperimentali attraverso simulazioni controllate, successivamente ogni componente dell'algoritmo viene testata e validata in modo incrementale. Particolare attenzione è stata posta alla riproducibilità degli esperimenti e alla tracciabilità dei risultati attraverso la generazione automatica di grafici comparativi e report quantitativi.

Nei successivi capitoli si parlerà dei fondamenti teorici dell'algoritmo ICP, le

scelte implementative effettuate, i risultati sperimentali ottenuti e i possibili sviluppi futuri. L'obiettivo finale è fornire un'analisi completa delle potenzialità e dei limiti dell'approccio ICP point to point nella localizzazione di robot in ambiente 2D, evidenziando quanto sia importante il filtraggio e l'ottimizzazione per poter raggiungere prestazioni affidabili in scenari applicativi realistici.

Capitolo 1

Fondamenti teorici e modello del sistema

Questo capitolo introduce i fondamenti teorici su cui si basa il sistema di simulazione sviluppato. Vengono presentati il modello cinematico del robot mobile, i principi di funzionamento del sensore LiDAR, il problema della localizzazione robotica e l'algoritmo ICP per allineare le nuvole di punti. Questi sono gli elementi che costituiscono la base necessaria per comprendere l'architettura e il funzionamento del simulatore che verrà descritto nei successivi capitoli.

1.1 Modello cinematico del robot mobile

Il robot mobile presente in questo progetto è stato modellato come un **uniciclo** (*unicycle model*), ovvero una delle rappresentazioni più diffuse nella robotica mobile. questo modello descrive un robot in grado di muoversi su un piano bidimensionale con due gradi di libertà che possono essere modificati: la velocità lineare (v) e la velocità angolare (ω). Alcune delle principali

assunzioni da dover fare per poter utilizzare il modello a uniciclo sono il fatto che il robot si muova senza alcun tipo di slittamento laterale e che il movimento sia vincolato alla direzione in cui è orientato, per questi motivi il modello è particolarmente adatto per veicoli a due ruote o con trazione anteriore/posteriore.

Lo stato del robot in un determinato istante di tempo (**posa**) è descritto da un vettore di tre elementi

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \alpha \end{bmatrix}$$

dove x e y rappresentano la posizione nel piano cartesiano e α l'orientamento rispetto all'asse x . Le leggi cinematiche che guidano il robot nella simulazione sono le seguenti:

$$\begin{cases} \dot{x} = v \cos(\alpha) \\ \dot{y} = v \sin(\alpha) \\ \dot{\alpha} = \omega \end{cases}$$

Possiamo osservare come quindi la velocità lineare determini lo spostamento nella direzione in cui è orientato il robot, mentre la velocità angolare controlli la variazione dell'angolo. Dunque la velocità lineare v proietta il movimento lungo l'asse longitudinale del robot attraverso le funzioni trigonometriche che dipendono dall'attuale orientamento.

Poiché il sistema opera attraverso equazioni continue ma la simulazione avviene in tempo discreto, è necessario utilizzare metodi di integrazione numerica. In questo progetto è stato adottato il metodo di **Eulero**, uno dei più diffusi, che dato un passo temporale Δt , lo stato al tempo $t + \Delta t$ è calcolato come

segue:

$$\begin{cases} x_{k+1} = x_k + \Delta t \cdot v_k \cos(\alpha_k) \\ y_{k+1} = y_k + \Delta t \cdot v_k \sin(\alpha_k) \\ \alpha_{k+1} = \alpha_k + \Delta t \cdot \omega_k \end{cases}$$

dove l'indice k indica il passo temporale discreto.

Questo schema viene applicato ad ogni istante di simulazione nella classe **Robot** per aggiornare la posa in base ai comandi di velocità correnti. Nonostante sia un metodo semplice è in grado di fornire risultati accurati per piccoli passi temporali ($\Delta t \approx 0.05 - 0.1$ secondi) e per velocità moderate. È quindi importante la scelta di Δt : valori troppo grandi possono introdurre errori di discretizzazione significativi, mentre valori troppo piccoli aumentano il carico computazionale senza però cambiamenti rilevanti.

1.2 Sensori LiDAR per la robotica mobile

Il sensore **LiDAR** (*Light Detection And Ranging*) è uno strumento essenziale e uno dei più diffusi nella robotica mobile. Il suo compito è quello di percepire l'ambiente circostante al robot sul quale è montato e il suo principio di funzionamento si basa sull'emissione di impulsi laser che, riflettendosi sugli ostacoli presenti nell'ambiente, permettono di misurare quanto sono distanti tali oggetti dal sensore stesso. In questo progetto, in cui l'ambiente sviluppato è bidimensionale, il LiDAR effettua le misurazioni sul piano orizzontale permettendo di acquisire le informazioni in un determinato intervallo angolare.

L'implementazione del sensore si basa sulla tecnica del **ray-casting**, cioè un algoritmo geometrico che va a simulare la diffusione dei raggi nell'ambiente.

Vengono tracciati dei raggi che partono dal centro del robot (punto in cui si è immaginato il piazzamento del sensore) e si estendono fino a una distanza massima predefinita ρ_{max} . Durante la propagazione l'algoritmo controlla se sono presenti intersezioni tra i raggi e gli ostacoli dell'ambiente sfruttando operazioni geometriche fornite dalla libreria **Shapely**. Nel momento in cui si rileva un'intersezione viene registrata la distanza del punto di impatto più vicino al sensore, ottenendo in questo modo la distanza associata ad una particolare direzione angolare.

Il sensore implementato è caratterizzato da parametri quali il **numero di raggi**, che definisce la risoluzione angolare delle scansioni, l'**ampiezza angolare**, che indica il campo visivo del sensore, e la **portata massima** oltre la quale il sensore non rileva più misurazioni.

Le scansioni del sensore LiDAR prendono il nome di **nuvole di punti** (*point cloud*). Ogni nuvola di punti è un piano cartesiano bidimensionale che rappresenta i punti rilevati nell'ambiente. Tali nuvole di punti possono essere espresse in due diversi sistemi di riferimento: **frame locale**, cioè quello legato al robot e centrato nella sua posizione, e **frame globale** che è il sistema assoluto dell'ambiente di simulazione.

È essenziale generare nuvole di punti accurate e dense in quanto la qualità e la ricchezza delle informazioni contenute nelle scansioni influenzano in modo diretto la precisione della stima del movimento del robot (e quindi la sua localizzazione) tramite tecniche come l'algoritmo ICP.

1.3 Localizzazione robotica

Come detto nell'introduzione, la localizzazione rappresenta uno dei problemi fondamentali della robotica mobile autonoma e tratta della capacità del

robot di saper determinare la propria posizione e orientamento rispetto a un dato sistema di riferimento. Nel progetto il problema si traduce nella stima della posa del robot in un ambiente 2D.

L'**odometria** è l'approccio più diretto per la localizzazione; sfrutta le informazioni sul movimento del robot per calcolare come cambia nel tempo la sua posizione e il suo orientamento. Come descritto nel dettaglio nel paragrafo 1.1, la posa viene aggiornata applicando le equazioni cinematiche con metodi di integrazione numerica; tuttavia si hanno limitazioni dovute all'accumulo progressivo di errori, fenomeno conosciuto come *drift odometrico*. Questa problematica assieme a imprecisioni nei parametri, errori di discretizzazione e incertezze nei parametri di controllo causano una divergenza sempre maggiore nella stima della posa del robot.

La soluzione a tutto ciò è ricorrere a sensori esterni che forniscono misure dirette dell'ambiente circostante. Il LiDAR è uno di tali sensori ed è attraverso il confronto di nuvole di punti consecutive che è possibile stimare il movimento relativo del robot. L'ICP è uno degli algoritmi che sono in grado di effettuare l'allineamento geometrico delle scansioni, ottenendo quindi una stima della posa più accurata e robusta.

1.4 Algoritmo ICP (Iterative Closest Point)

Il principio alla base dell'algoritmo **ICP** consiste nel trovare la trasformazione rigida ottimale che allinea due nuvole di punti generate da scansioni consecutive, minimizzando la distanza tra le corrispondenze degli stessi punti. Nel progetto, avendo un contesto bidimensionale, la trasformazione rigida è definita da una matrice di rotazione $\mathbf{R}$ (2x2) e un vettore di traslazione $\mathbf{t}$ (2D) che se composte insieme definiscono il movimento relativo del robot.

ICP è un algoritmo **iterativo** basato su due passi fondamentali ripetuti fino a convergenza. Il primo è l'**associazione delle corrispondenze** (*nearest neighbor matching*), in cui per ogni punto della nuvola sorgente viene identificato il punto più vicino nella nuvola target. Il risultato è un insieme di coppie di punti corrispondenti tra le due scansioni consecutive, le quali rappresentano lo stesso elemento geometrico dell'ambiente scansionato da posizioni differenti. Il secondo passo dell'ICP è la **stima della trasformazione ottimale** che va a minimizzare l'errore quadratico medio (**RMSE**) tra i punti sorgente e target associati. Questa operazione è risolta sfruttando la decomposizione ai valori singolari (**SVD**), un metodo di algebra lineare che permette di ottenere una soluzione precisa al problema di allineamento. La convergenza viene gestita tramite l'osservazione della variazione dell'errore RMSE tra iterazioni consecutive: nel momento in cui l'errore scende sotto una soglia stabilita (*tolerance*) l'algoritmo termina restituendo la trasformazione stimata. Importante porre attenzione sul fatto che ICP è un algoritmo che opera a livello locale, ciò vuol dire che la sua efficacia dipende fortemente dall'inizializzazione: una stima iniziale della trasformazione molto vicina a quella reale migliora significativamente la robustezza e quindi la stima della posa, è questa la parte in cui viene sfruttata l'odometria in modo da evitare di convergere verso minimi locali errati.

Nel Capitolo 3 verranno analizzate in dettaglio le diverse strategie di inizializzazione implementate e il loro impatto sulle prestazioni complessive della simulazione.

Capitolo 2

Implementazione del sistema di simulazione

In questo capitolo si parla dell'implementazione in modo dettagliato della simulazione sviluppata. Il simulatore è composto da tutti gli elementi necessari a riprodurre un sistema robotico mobile reale: il robot con la sua cinematica, l'ambiente operativo con ostacoli di varia natura, il sensore LiDAR per la percezione dell'ambiente circostante, e un generatore di traiettorie per il movimento pianificato. Grazie all'architettura adottata è possibile configurare facilmente diversi scenari sperimentali andando a variare i parametri del robot, le caratteristiche del sensore, la complessità dell'ambiente e le tipologie di movimento. Attraverso questa flessibilità è così possibile testare le prestazioni dell'algoritmo ICP. Vengono di seguito descritti in dettaglio l'implementazione e le scelte progettuali.

2.1 Architettura del simulatore

Il progetto adotta un'architettura modulare che garantisce flessibilità e manutenibilità del codice. Grazie a questa divisione è possibile introdurre nuove funzionalità o modificare comportamenti esistenti poiché le modifiche restano confinate ai moduli interessati.

Il sistema si articola in cinque moduli principali che interagiscono attraverso interfacce ben definite. Il modulo **robot.py** gestisce lo stato cinematico e l'aggiornamento della posa in base ai comandi di velocità ricevuti, inoltre espone i metodi per impostare i comandi (v e ω). Il modulo **environment.py** è responsabile della rappresentazione dello spazio operativo e degli ostacoli e fornisce metodi per aggiungere ostacoli di diverse tipologie, verificare collisioni con pose specifiche, e calcolare intersezioni tra raggi e geometrie, operazioni fondamentali per il corretto funzionamento della simulazione. Il modulo **lidar.py** implementa la logica del sensore mediante *ray-casting*, riceve in ingresso la posa corrente del robot e l'ambiente e restituisce nuvole di punti che rappresentano le letture del sensore. Il modulo **trajectory_simulator.py** produce sequenze di comandi di velocità corrispondenti alle traiettorie predefinite. Il modulo **icp.py** contiene l'algoritmo ICP e tutti i suoi metodi ausiliari che gestiscono i filtri per migliorare il risultato ottenuto. Infine, il modulo **main.py** orchestra l'esecuzione complessiva.

Le interazioni seguono un flusso ben definito: ad ogni step di simulazione, il **TrajectoryGenerator** fornisce i comandi di velocità al **Robot**, che aggiorna la propria posa. Il **Lidar**, interrogando **Environment** e utilizzando la posa aggiornata, genera una nuova nuvola di punti. I dati raccolti vengono poi utilizzati per l'algoritmo **ICP** e per la generazione di visualizzazioni.

2.2 Modellazione del robot

La classe **Robot** contiene l'implementazione del modello cinematico *unicycle*, qui troviamo lo stato del robot e i relativi metodi di aggiornamento e interrogazione. Lo stato è rappresentato tramite un array *NumPy* di tre elementi che corrispondono alla tripletta $[x, y, \alpha]$, inizializzato alla posa di partenza specificata. La classe mantiene anche lo storico completo delle pose attraversate per poter generare le visualizzazioni della traiettoria, analizzare il comportamento del robot e confrontare traiettorie stimate con quelle reali. Tra gli attributi della classe troviamo le velocità lineari e angolari massime ammissibili utilizzate per verificare che i comandi ricevuti siano realistici. Il metodo principale **update(v, alpha, dt)** implementa l'integrazione numerica delle equazioni cinematiche descritte nella sezione 1.1 usando il metodo di Eulero. L'angolo α è normalizzato nell'intervallo $[-\pi, \pi]$ dopo ogni aggiornamento per prevenire problemi numerici nelle trasformazioni successive. La classe contiene anche metodi getter per accedere alla posa corrente, all'orientamento e alla traiettoria completa e i metodi setter per i comandi di velocità. Il metodo **reset()** permette di reinizializzare la posa del robot ad una configurazione iniziale specifica, essenziale per eseguire simulazioni consecutive con parametri diversi mantenendo condizioni iniziali identiche.

2.3 Ambiente e gestione degli ostacoli

L'ambiente della simulazione è implementato tramite l'uso della libreria **Shapely**, una libreria Python per la gestione e l'analisi di geometrie planari. Shapely fornisce metodi per effettuare operazioni come intersezioni, unioni e calcoli di distanze. Tramite l'uso di questa libreria si può così evitare di implementare algoritmi geometrici da zero.

Principalmente sono stati utilizzati **Point** per rappresentare punti, **LineString** per disegnare linee spezzate e **Polygon** per i gli ostacoli poligonali.

Nel progetto sono stati utilizzati quattro tipi di ostacoli:

- **Rettangoli:** definiti specificando le coordinate dei due vertici opposti (x_min, y_min) e (x_max, y_max) . Sono ideali per rappresentare pareti ortogonali alla traiettoria.
- **Cerchi:** specificati tramite centro (c_x, c_y) e raggio r . Dato che Shapely non supporta nativamente i cerchi esatti, questi vengono approssimati come poligoni regolari con un elevato numero di lati.
- **Poligoni arbitrari:** tramite una lista ordinata di vertici rendono possibile la rappresentazione di forme complesse e irregolari. Shapely gestisce automaticamente la chiusura del poligono unendo l'ultimo vertice al primo della lista.
- **Muri:** rappresentati come LineString, cioè segmenti senza area interna. utilizzati per definire i confini dell'ambiente o barriere sottili. Sono definiti tramite due punti in caso di segmento singolo oppure una sequenza di punti se si vuole disegnare una linea spezzata.

La classe Environment si occupa inoltre delle strategie di piazzamento di tali ostacoli permettendo di generare ambienti standardizzati o casuali. Le strategie utilizzate si occupano di distribuire in modo casuale uniforme gli elementi dell'ambiente in modo tale che non si sovrappongano tra di loro o che intersechino la traiettoria del robot (operazione svolta da metodi helper contenuti in questa classe), in questo modo è possibile valutare la robustezza della simulazione.

2.4 Generazione delle traiettorie

Nel progetto sono state implementate quattro tipologie di traiettorie:

- **Traiettorie rettilinee:** rappresentano il caso più semplice, il robot si muove in linea retta con velocità angolare ω nulla e velocità lineare in un caso costante e nell'altro variabile. Queste traiettorie sono utili per verificare il comportamento base dell'ICP.
- **Traiettorie circolari:** hanno velocità angolare ω costante e velocità lineare costante o variabile a seconda del caso. Il rapporto v/ω determina il raggio di curvatura del percorso e il segno di ω definisce il senso di percorrenza, se positivo il verso è antiorario mentre è orario se il segno è negativo. Queste traiettorie sono fondamentali per controllare il comportamento di ICP in presenza di rotazioni significative.
- **Traiettorie a forma di otto:** combina due archi circolari di verso opposto (si ha quindi un'inversione del segno di ω) che si intersecano nel punto centrale. Traiettorie interessanti poiché il robot attraversa più volte la stessa regione da direzioni diverse.
- **Traiettorie casuali:** in questo caso i comandi di velocità vengono perturbati casualmente entro range predefiniti. Ad ogni step i comandi vengono modificati aggiungendo rumore gaussiano o uniforme con deviazione standard configurabile. Grazie a dei vincoli i comandi di velocità restano entro limiti fisici realistici. La traiettoria casuale ha come obiettivo quello di simulare un possibile percorso reale in cui il robot affronta ostacoli imprevisti o è in presenza di disturbi esterni.

Ogni tipologia di traiettoria è guidata da una serie di parametri che ne determinano le caratteristiche. Comuni a tutte le traiettorie sono le **velocità**

nominali che definiscono i comandi base, la **durata** espressa come tempo totale oppure come distanza totale da percorrere (è il sistema a calcolare il numero di step necessari). Per le traiettorie curve sono poi presenti i parametri quali il **raggio di curvatura** mentre per la traiettoria casuale ci sono in aggiunta l'**ampiezza del rumore** e il **tipo di distribuzione**. Un parametro importante è la **Frequenza di campionamento** che determina ogni quanti step i comandi vengono aggiornati: a frequenze alte corrispondono traiettorie più smussate (*smoothed*) e viceversa.

La classe **TrajectoryGenerator** gestisce la logica di generazione e fornisce un'interfaccia unica per tutte le tipologie di traiettoria. Quando viene inizializzata si specifica il tipo di traiettoria desiderato e si passano tutti i parametri specifici di tale caso. Grazie al metodo `get_command(t)` si recupera la coppia di comandi (v, ω) appropriata per l'istante temporale t e adeguata alla traiettoria richiesta.

La classe mantiene traccia della fase corrente della traiettoria (in quale lobo dell'otto si trova il robot o lo stato del random number generator per la traiettoria casuale). Si trova anche un metodo `reset()` che permette di reinizializzare lo stato per ripetere la stessa traiettoria.

2.5 Simulazione del sensore LiDAR

Il sensore LiDAR viene implementato nel progetto tramite la tecnica del **ray-casting**. Viene generato un ventaglio di raggi uniformemente distribuiti in una determinata apertura angolare, di seguito viene calcolata per ciascun raggio l'intersezione con gli ostacoli e poi determinata la distanza al punto d'impatto più vicino.

L'implementazione è la seguente: dato il numero di raggi n_rays e l'apertura angolare totale $angle_span$, si calcola la spaziatura angolare tra raggi consecutivi come $\Delta\alpha = angle_span / (n_rays - 1)$. Per ogni raggio i -esimo, si calcola l'angolo $\alpha_i = -angle_span/2 + i \cdot \Delta\alpha$ relativo all'asse longitudinale del robot. L'angolo assoluto nel frame globale è $\theta_global = \theta_robot + \alpha_i$, dove θ_robot è l'orientamento corrente del robot.

Per ogni raggio si costruisce un **LineString** di Shapely che parte dalla posizione del robot (x_r, y_r) e si estende fino alla distanza massima r_max nella direzione determinata da θ_global . Il punto finale del raggio è $(x_r + r_max \cdot \cos(\theta_global), y_r + r_max \cdot \sin(\theta_global))$. Quindi viene calcolata l'intersezione del **LineString** ottenuto con gli ostacoli dell'ambiente e viene restituito un **Point** o un **MultiPoint**, in tal caso si seleziona il punto più vicino al robot. Se non sono presenti intersezioni entro **r_max** il raggio non restituisce nessuna misura.

Il sensore implementato viene gestito principalmente dai seguenti tre parametri:

- **Numero di raggi:** definisce la risoluzione angolare del sensore. Nel progetto è impostato a 360 in modo tale che sia presente un raggio per grado (sensore *full-range*). Aumentare il numero di raggi migliora la qualità della nuvola di punti ma aumenta linearmente il costo computazionale.
- **Apertura angolare:** determina l'ampiezza della scansione ed è espressa in radianti. Nel progetto ha il valore di 2π in modo da realizzare un sensore *omnidirezionale*, sensori con apertura angolare minore si definiscono invece *direzionali*. Si è scelto il sensore omnidirezionale

in modo tale da doverne implementare solamente uno senza perdere informazioni spaziali dovute ad aree non coperte.

- **Distanza massima:** è il range operativo del sensore, oltre il quale gli ostacoli non vengono rilevati. La scelta di tale lunghezza deve essere bilanciata poiché valori troppo elevati rendono visibili ostacoli irrilevanti, mentre valori troppo piccoli generano scansioni prive di dettagli utili alla navigazione.

Le nuvole di punti prodotte dal sensore LiDAR possono essere **locali** o **globali**. Nel frame locale del robot i punti rilevati sono rappresentati rispetto alla posa corrente del sensore. Questo è il sistema di riferimento naturale per il sensore e per l'algoritmo ICP che opera confrontando scansioni locali successive in modo tale da non dover conoscere la posa globale del robot.

Nel frame globale i punti vengono quindi trasformati applicando la trasformazione rigida corrispondente alla posa del robot: data la posa (x_r, y_r, θ_r) e un punto in coordinate locali (x_l, y_l) le coordinate globali sono

$$\begin{cases} x_{global} = x_r + x_l \cdot \cos(\theta_r) - y_l \cdot \sin(\theta_r) \\ y_{global} = y_r + x_l \cdot \sin(\theta_r) + y_l \cdot \cos(\theta_r) \end{cases}$$

Questa trasformazione è una rotazione di θ_r seguita da una traslazione di (x_r, y_r) . La rappresentazione in frame globale è essenziale per visualizzazioni della mappa complessiva, per confronti con ground truth, e per validazione degli algoritmi di localizzazione. Per questo motivo vengono generate entrambe le rappresentazioni a ogni step, in questo modo le nuvole locali sono passate a ICP per il calcolo della stima della posa tra istanti di tempo successivi, mentre le nuvole globali sono utilizzate per valutare l'errore di localizzazione e per produrre visualizzazioni. La presenza di entrambe

le rappresentazioni è fondamentale per un'analisi completa del sistema: permette di distinguere errori nell'algoritmo ICP (che opera su dati locali) da errori nell'integrazione della traiettoria (che si verificano nel confronto con *ground truth* globale).

Capitolo 3

Algoritmo ICP e ricostruzione della posa

In questo capitolo si tratterà l'implementazione dell'algoritmo **Iterative Closest Point (ICP)** utilizzato nel progetto per stimare il movimento relativo del robot tra scansioni LiDAR consecutive. Si parlerà della pipeline dell'algoritmo, delle diverse strategie di inizializzazione basate su odometria o meno, delle tecniche per la ricostruzione della traiettoria mediante composizione incrementale delle trasformazioni e, infine, si tratterà delle metriche utilizzate per valutare le prestazioni di ICP in termini di accuratezza e robustezza.

3.1 Implementazione dell'algoritmo ICP

Nel progetto è stato utilizzato l'approccio ICP **point to point** che alterna due operazioni fondamentali: l'associazione delle corrispondenze tra punti delle due nuvole e il calcolo della trasformazione rigida ottimale che minimizza l'errore di allineamento. Complessivamente la pipeline è composta da

un'inizializzazione, un ciclo iterativo e un criterio di convergenza. Vediamoli nel dettaglio:

- **Inizializzazione:** l'algoritmo riceve in ingresso due nuvole di punti espresse nel frame locale del robot. Una nuvola si dice **source** e corrisponde alla scansione all'istante temporale k , l'altra si dice **target** e si riferisce alla scansione precedente all'istante $k - 1$. In questa fase è possibile fornire una stima iniziale della trasformazione tramite una matrice di rotazione R_{init} e un vettore di traslazione t_{init} , se non vengono specificati allora ICP parte dalla trasformazione **identità** ($R = I, t = 0$). Tramite questa stima iniziale si ottiene così una prima approssimazione dell'allineamento.
- **Ciclo iterativo:** ogni iterazione esegue il **nearest neighbor matching**, cioè per ogni punto della nuvola source trasformata si identifica il punto più vicino nella nuvola target. Il risultato è un insieme di coppie di punti corrispondenti (p_{source}, p_{target}) . Per via della presenza di punti troppo distanti (*outlier*) c'è la possibilità che avvengano associazioni errate, per questo, per aumentare la robustezza dell'algoritmo, si applica la soglia di distanza massima **max_correspondence_distance**: le coppie con distanza superiore a questa vengono scartate. Ottenute le corrispondenze, viene calcolata la trasformazione ottimale mediante decomposizione ai valori singolari (**SVD**). Il procedimento inizia calcolando i centroidi delle due nuvole:

$$c_{source} = \frac{\sum p_{source_i}}{N}$$
$$c_{target} = \frac{\sum p_{target_i}}{N}$$

con N numero di corrispondenze valide. Successivamente vengono centrate le nuvole andando a sottrarre i rispettivi centroidi ottenendo così

$\tilde{p}_{source}$ e $\tilde{p}_{target}$. Poi si calcola la matrice di covarianza:

$$H = \sum \tilde{p}_{target_i} \cdot (p_{source_i})^T$$

Da qui si effettua la sua decomposizione SVD che fornisce:

$$H = U \sum V^T$$

da cui si ricava la rotazione ottimale:

$$R = UV^T$$

Si deve inoltre verificare che $\det(R) = +1$ per assicurare che sia una rotazione propria e non una riflessione, altrimenti si procede a invertire il segno dell'ultima colonna di U prima di fare il prodotto. Infine la traslazione ottimale si ottiene come segue:

$$t = c_{target} - R \cdot c_{source}$$

Di seguito la trasformazione appena calcolata viene applicata alla nuvola source per aggiornarne l'allineamento e contemporaneamente viene composta anche con la trasformazione cumulativa tenuta in memoria dall'inizio dell'algoritmo. Questo perché ad ogni passo viene stimato un piccolo aggiustamento incrementale ma la trasformazione totale che allinea la nuvola source originale alla target è data da:

$$R_{total} = R_{iter} \cdot R_{total}$$

$$t_{total} = R_{iter} \cdot t_{total} + t_{iter}$$

- **Criterio di convergenza:** per monitorare la convergenza si usa l'**errore quadratico medio (RMSE)** calcolato tra i punti corrispondenti dopo l'applicazione della trasformazione. Se la variazione di RMSE tra due

iterazioni consecutive scende sotto la soglia predefinita **tolerance** l'ICP termina dopo aver raggiunto la convergenza. In caso contrario vengono ripetute altre iterazioni fino a un massimo di **max_iterations** (per evitare cicli infiniti).

Il risultato finale di questa pipeline è un dizionario contenente la trasformazione stimata (R, t) , il numero di iterazioni eseguite, l'RMSE finale, l'angolo di rotazione in gradi e l'andamento dell'errore durante le iterazioni.

3.2 ICP con e senza inizializzazione odometrica

ICP è fortemente influenzato dalla stima iniziale della trasformazione, per questo nel progetto sono state implementati due diversi approcci: uno **ICP raw** che parte dalla trasformazione identità senza informazioni preliminari, l'altro **ICP filtrato** che utilizza una stima odometrica iniziale per velocizzare e migliorare la convergenza.

Nel caso di **ICP raw** la trasformazione viene inizializzata con $R = I$ (matrice identità 2x2) e $t = 0$ (vettore nullo), si presume quindi che il robot non si sia mosso o lo abbia fatto di pochissimo tra le due scansioni consecutive. Questo tipo di approccio è indipendente dall'odometria e si basa esclusivamente sulle informazioni geometriche ottenute dal sensore LiDAR. Il vantaggio principale è ovviamente che non è soggetto a errori dovuti al drift dell'odometria, per cui ICP raw può in alcuni casi fornire stime più affidabili. Sono presenti però alcune limitazioni: se il robot effettua movimenti importanti oppure l'ambiente ha simmetrie geometriche l'algoritmo può generare trasformazioni errate. Inoltre, ICP raw necessita di un numero di iterazioni maggiore rispetto a ICP filtrato.

ICP filtrato utilizza invece un'inizializzazione basata su odometria del robot. Quello che accade è che prima di eseguire ICP si calcola la trasformazione relativa prevista dall'odometria tra le nuvole di punti agli istanti k e $k-1$ tramite il metodo `compute_relative_transform_from_odometry(prev_pose, curr_pose)`. Gli input sono dunque le pose del robot nei due istanti consecutivi $(x_{k-1}, y_{k-1}, \alpha_{k-1})$ e (x_k, y_k, α_k) ; da questi si calcola poi la trasformazione nel frame locale del robot. L'angolo di rotazione viene ricavata dalla differenza angolare $\Delta\alpha = \alpha_k - \alpha_{k-1}$, da qui si costruisce la matrice di rotazione:

$$R_{odom} = [[\cos(\Delta\alpha), -\sin(\Delta\alpha)], [\sin(\Delta\alpha), \cos(\Delta\alpha)]]$$

Per ottenere il vettore di traslazione si calcola prima lo spostamento nel frame globale:

$$\Delta x_{world} = x_k - x_{k-1}$$

$$\Delta y_{world} = y_k - y_{k-1}$$

che viene poi trasposto nel frame locale tramite rotazione inversa:

$$t_x = \cos(\alpha_{k-1}) \cdot \Delta x_{world} + \sin(\alpha_{k-1}) \cdot \Delta y_{world}$$

$$t_y = -\sin(\alpha_{k-1}) \cdot \Delta x_{world} + \cos(\alpha_{k-1}) \cdot \Delta y_{world}$$

Otengo così la trasformazione (R_{odom}, t_{odom}) che viene utilizzata come inizializzazione di ICP. Tra i vantaggi di questo approccio troviamo la convergenza più rapida perché si parte da una stima vicina alla soluzione reale, si riduce il rischio di convergenza verso minimi locali e quindi errati e inoltre si ha più robustezza in ambienti privi di elementi distintivi. Tuttavia va tenuto conto della presenza del drift odometrico che può portare a una stima iniziale distorta penalizzando ICP.

Nel metodo `run_icp_pair` vengono eseguite entrambe le varianti sulla stessa coppia di scansioni, ottenendo come risultato un dizionario che contiene i dettagli di entrambi gli approcci. Questo permette di analizzare quale strategia sia più efficace in funzione delle caratteristiche della traiettoria, della densità degli ostacoli e della qualità dell'odometria.

3.3 Ricostruzione della traiettoria

Dal momento in cui l'obiettivo finale del progetto è ricostruire l'intera traiettoria del robot nel tempo è necessario un altro passaggio dopo aver calcolato le stime con ICP. Ciò che deve essere fatto è comporre le trasformazioni stimate in modo da ottenere la sequenza completa delle pose nel frame globale dell'ambiente.

Il processo parte dalla posa iniziale al tempo $k = 0$ (nota e utilizzata come riferimento). Per ogni passo successivo ICP stima la trasformazione relativa (R_k, t_k) nel frame locale della posa precedente, questa descrive il movimento fatto dal robot tra $k - 1$ e k nel sistema di riferimento centrato sulla posa al tempo $k - 1$, non nel frame globale.

Per ottenere la posa globale nel momento k si deve comporre la trasformazione stimata con la posa globale al tempo $k - 1$. Questo si fa convertendo il vettore di traslazione t_k dal frame locale al globale attraverso rotazione (in base all'orientamento corrente del robot) e poi aggiornando la posizione globale sommando questa traslazione trasformata alla posizione precedente. Invece per l'orientamento, la rotazione R_k viene composta con la matrice di rotazione globale precedente e l'angolo viene estratto dalla matrice risultante con la funzione `arctan2`, normalizzandolo nell'intervallo $[-\pi, \pi]$ per evitare

accumulo di errori numerici.

Questo procedimento viene ripetuto per ogni passo temporale ottenendo così la sequenza completa delle pose stimate. L'aspetto problematico è l'accumulo degli errori nel tempo: ogni trasformazione stimata da ICP ha al suo interno un piccolo errore dovuto a rumore nelle misure, approssimazioni numeriche e altri limiti dell'algoritmo. Componendo tra loro queste trasformazioni inevitabilmente tali errori si sommano tra loro producendone uno più rilevante che porta a un drift nella traiettoria stimata rispetto alla reale. Questo fenomeno è analogo al drift odometrico ma meno evidente per via delle correzioni date dalle scansioni prodotte dal sensore LiDAR.

3.4 Metriche di valutazione

Nel progetto sono state definite diverse **metriche** per poter valutare le prestazioni dell'algoritmo ICP riguardo l'accuratezza, l'efficienza e la robustezza. Vediamole più nel dettaglio:

- **RMSE** (Root Mean Square Error): è la metrica principale utilizzata per verificare la qualità dell'allineamento, osserva la discrepanza media tra i punti della nuvola source trasformata e i corrispondenti punti della nuvola target. L'RMSE viene calcolato dopo aver applicato la trasformazione stimata (R, t) e aver individuato le N coppie corrispondenti:

$$\mathbf{RMSE} = \sqrt{\frac{\sum_{i=1}^N ||R \cdot p_{source_i} + t - p_{target_i}||^2}{N}}$$

RMSE fornisce quindi una misura in metri dell'errore di allineamento e identifica la precisione con cui ICP è riuscito a sovrapporre le due nuvole

di punti. Valori vicini allo zero indicano un allineamento ben riuscito e viceversa. Da notare come i punti scartati (outlier) non influiscono sul calcolo di RMSE. Inoltre, questa metrica, come già detto nel paragrafo 3.1, viene utilizzata per verificare la convergenza di ICP: l'algoritmo si ferma quando il valore di RMSE tra iterazioni consecutive scende sotto una soglia di tolleranza.

- **Numero di iterazioni:** conta quante iterazioni sono state fatte prima di giungere a convergenza. Questa metrica valuta l'efficienza computazionale dell'algoritmo ICP. Nel confronto tra ICP raw e ICP filtrato il numero di iterazioni è molto differente: nella versione filtrata, per via dell'inizializzazione odometrica, si hanno tipicamente tra le 5 e le 15 iterazioni, mentre nel caso raw le iterazioni salgono a 20 - 40.
- **Numero di corrispondenze valide:** misura ottenuta durante il nearest neighbor matching, fornisce un'indicazione di quanto due scansioni siano state sovrapposte correttamente. Un alto numero di corrispondenze indica una buona sovrapposizione geometrica e di conseguenza una stima della trasformazione robusta. L'algoritmo è strutturato in modo tale da rifiutare coppie di scansioni con meno di tre corrispondenze valide.
- Sono poi presenti altre metriche come errore medio, errore massimo, deviazione standard e drift finale che osservano errori di posizione e di orientamento rispetto alla ground truth. L'analisi di queste metriche, tramite grafici, permette di individuare fasi critiche dell'algoritmo ICP in casi specifici.

Queste analisi quantitative sono la base di partenza per la validazione sperimentale presentata nel capitolo successivo.

Capitolo 4

Validazione sperimentale

Capitolo 5

Conclusioni e sviluppi futuri

Bibliografia